GitHub Copilot

Step-by-Step Prompt Guide

AI Interview Module — Complete Implementation

Model to Use	Claude Sonnet 4.6 (see Section 1 for why)
Total Prompts	18 ready-to-paste prompts across 6 phases
Estimated Time	3-5 days of focused coding with Copilot
Reference File	AI_Interview_Blueprint.docx (attach to every chat)
Target Output	Industry-level voice interview module, zero bugs

YSPM's Yashoda Technical Campus, Satara | AI & DS Department | 2025-26

1. Which Model to Use — and Why

You have a GitHub Student Pro account. Based on the model list in your screenshot, here is the exact model to select and why it is the best choice for this specific project.

✅SELECT THIS MODEL: Claude Sonnet 4.6 (1x cost)

In the Copilot model dropdown, select: Claude Sonnet 4.6

Do NOT use Claude Opus 4.6 for coding — it costs 3x more credits and is slower.

Sonnet 4.6 is specifically optimized for code generation. It is the same tier that powers Claude.ai's code tasks.

Your GitHub Student Pro credits will last 3x longer compared to Opus.

Model	Cost	Best For	Verdict for Your Project
Claude Haiku 4.5	0.33x	Quick completions, simple edits	❌Too weak for complex state machine code
Claude Opus 4.5 / 4.6	3x	Research, reasoning, analysis	❌Overkill for coding — wastes credits fast
Opus 4.6 fast (Preview)	30x	Experimental use only	❌Never use this — burns all credits instantly
Claude Sonnet 4 / 4.5	1x	Good coding, older version	☐ Works but 4.6 is better
Claude Sonnet 4.6	1x	Production-level code generation	✅BEST CHOICE — use this for everything

How to Set It in VS Code

1. Open VS Code → Click the Copilot icon (bottom right) OR press Ctrl+Shift+I to open Copilot Chat
2. In the chat panel, click the model name at the bottom (currently shows 'Claude Opus 4.6')
3. Select 'Claude Sonnet 4.6' from the dropdown
4. Keep this model selected for ALL 18 prompts in this guide
5. In the bottom bar, also set 'Edit automatically' — this lets Copilot edit files directly

2. How to Use This Guide — Rules to Follow

Follow these rules for every single prompt. They are the difference between Copilot writing perfect code vs writing broken code.

Rule #	Rule	Why It Matters
1	Always attach the blueprint file. Before sending Prompt 1, drag AI_Interview_Blueprint.docx into the Copilot chat window.	Copilot gets the full architecture context — database schema, API shapes, hook names — without you repeating it
2	Open the relevant file before sending the prompt. If the prompt is about Interview.jsx, have that file open in VS Code.	Copilot uses the open file as primary context. @workspace alone is not enough for complex files.
3	One prompt = one file or one function. Never ask for two different files in one prompt.	Smaller scope = better, more accurate code. Large prompts produce spaghetti.
4	After each prompt, read the generated code before accepting. Check: does it match the blueprint's interface?	Copilot sometimes simplifies. Your blueprint has exact prop names, state names, and function signatures — verify them.
5	If the output is wrong, do NOT send a new prompt. Use the Regenerate button first.	Sending a correction prompt without regenerating creates conflicting context that confuses the model.
6	Use Copilot Edits (not Chat) for writing new files. Press Ctrl+Shift+I → click 'Edits' tab.	Edits mode writes directly to files. Chat mode only shows code in a panel — you have to copy manually.
7	Always verify imports at the top of every generated file.	Copilot sometimes imports from non-existent paths. Fix all import paths before running the code.

3. Phase 1 — Project Setup & File Structure

These prompts create the folder structure and install dependencies. Run them in order before writing any logic code.

3.1 Before You Start — Attach Context to Copilot

Do this ONCE before sending any prompt

Step 1: Open GitHub Copilot Chat in VS Code (Ctrl + Shift + I)
Step 2: Switch model to Claude Sonnet 4.6
Step 3: Drag and drop AI_Interview_Blueprint.docx into the chat input box
Step 4: Type this exactly and press Enter:

'I have attached the engineering blueprint for my AI interview module.
Read and remember the full architecture. Do not write any code yet.
Just confirm you have understood the file structure, hook names,
API endpoints, and database schema from the document.'

Step 5: Wait for Copilot to confirm it has read the document.
Step 6: Now proceed with Prompt 1 below.

PROMPT 1 — Create All Empty Frontend Files

```
@workspace

Create the following empty files with the correct folder structure
inside the src/ directory of this React project.
Do NOT write any code inside them yet — just create the files
with a single comment showing the file's purpose.

Files to create:

src/pages/Interview.jsx
    // Main interview controller — state machine

src/pages/InterviewResults.jsx
    // Results dashboard + PDF download

src/components/interview/BrowserCheck.jsx
src/components/interview/MicPermission.jsx
src/components/interview/PreBrief.jsx
src/components/interview/QuestionCard.jsx
```

```
src/components/interview/AnswerReview.jsx
src/components/interview/LoadingScreen.jsx
src/components/interview/StatusBadge.jsx
src/components/interview/CountdownTimer.jsx

src/components/results/ScoreGauge.jsx
src/components/results/QuestionFeedbackCard.jsx
src/components/results/ReportTemplate.jsx

src/hooks/useTTS.js
src/hooks/useSTT.js
src/hooks/useSilenceDetector.js
src/hooks/useInterviewSession.js

src/services/api/interview.js

src/utils/pdfGenerator.js
src/utils/browserSupport.js
```

TIP: After this, do NOT add any logic. Just confirm all files are created.

PROMPT 2 — Install Required npm Packages

```
I need to install the following npm packages for my React project.
Generate the exact terminal commands to run, one by one:

1. jspdf          — for generating PDF reports client-side
2. html2canvas    — for capturing the results div as an image for PDF
3. chart.js       — for the radar chart in the results dashboard
4. react-chartjs-2 — React wrapper for Chart.js

Also generate the install command for the backend (Python/Flask):
5. google-generativeai  — Gemini AI SDK (replaces OpenAI)

Show me each command separately so I can run them one at a time.
Use npm install (not yarn). Use pip install for Python.
```

TIP: Run each command in the VS Code terminal (Ctrl+`) before moving to Phase 2.

PROMPT 3 — Create Backend File Structure

```
@workspace

In the backend/ folder of this Flask project, create these
empty Python files with a single docstring comment each:

backend/services/openai_service.py
    # Gemini AI API calls — question generation and interview evaluation

backend/services/prompt_builder.py
    # Builds GPT/Gemini prompts for question generation and evaluation

backend/services/resume_parser.py
    # Parses PDF/DOCX resumes using spaCy and PyPDF2

backend/utils/validators.py
    # Input validation helper functions

Do NOT write any logic — just the docstring comment in each file.
Also show me the complete backend/ folder structure as a tree
after creating the files.
```

4. Phase 2 — Build the Custom Hooks (The Engine)

These are the four most important files in the entire frontend. Build them first, test each in isolation, and only then build any UI components. Every component depends on these hooks.

PROMPT 4 — Build useTTS.js — Text-to-Speech Hook

```
@workspace  Open file: src/hooks/useTTS.js

Write the complete useTTS custom hook. Requirements:

1. Uses window.speechSynthesis (browser built-in, no library needed)
2. Exports one function: speak(text) → returns a Promise that resolves
   when the speech finishes (using utterance.onend)
3. Exports one function: stopSpeaking() — cancels ongoing speech
4. Rate: 0.88, Pitch: 1.0, Volume: 1.0
5. CRITICAL: Add a setInterval keep-alive ping every 5000ms that calls
   speechSynthesis.resume() if paused — this fixes Chrome's bug where
   long sentences pause mid-speech
6. Voice selection priority:
   → First pick: any voice containing 'Google' AND lang starts with 'en'
   → Second pick: lang starts with 'en-IN'
   → Third pick: any lang starting with 'en'
   → Fallback: default voice
7. Handle the case where getVoices() returns empty on first call
   (use speechSynthesis.onvoiceschanged event)
8. Cleanup: cancel speech on component unmount (useEffect cleanup)

IMPORTANT: The speak() function must clear the keep-alive interval
inside BOTH onend AND onerror handlers to prevent memory leaks.
```

TIP: After writing, TEST IT: In any component, call speak('Hello, welcome to your interview') and verify you hear audio.

PROMPT 5 — Build useSTT.js — Speech-to-Text Hook

```
@workspace  Open file: src/hooks/useSTT.js

Write the complete useSTT custom hook. Requirements:

Returns these values: { transcript, isRecording, isSupported,
  sttError, startRecording, stopRecording, resetTranscript }
```

```
1. On mount: set isSupported = false if window.SpeechRecognition
   AND window.webkitSpeechRecognition are both undefined
2. startRecording() must:
   - Set recognition.lang = 'en-IN'
   - Set recognition.continuous = true
   - Set recognition.interimResults = true
   - Build transcript by APPENDING new final results to
transcriptRef.current
   - Show live transcript (interim + final combined) via setTranscript()
   - Auto-restart recognition in the onend handler IF isRecordingRef is
true
     (this handles Chrome's 60-second auto-stop)
3. stopRecording() must:
   - Set isRecordingRef.current = false BEFORE calling recognition.stop()
   - Return the final transcript string (from
transcriptRef.current.trim())
4. Error handling — these errors must be IGNORED (not shown to user):
   'no-speech', 'aborted'
   All other errors: set sttError to the error code string
5. resetTranscript() clears both the state AND the ref
6. Cleanup on unmount: set ref to false, call stop() with try/catch

Use useRef for isRecordingRef and transcriptRef (NOT useState)
so that the onend callback always reads the latest values.
```

PROMPT 6 — Build useSilenceDetector.js

```
@workspace  Open file: src/hooks/useSilenceDetector.js

Write the useSilenceDetector hook. It is very simple but critical.

Accepts: (onSilence, silenceMs = 3000)
Returns: { startSilenceDetection, stopSilenceDetection, resetSilenceTimer
}

Logic:
- startSilenceDetection(): set activeRef=true, start the timer
- stopSilenceDetection(): set activeRef=false, clearTimeout
- resetSilenceTimer(): if activeRef is true, clear the existing
  timeout and start a new one for silenceMs milliseconds.
  When the timer fires: call onSilence() ONLY if activeRef is still true.

Use useRef for the timer and active flag (NOT useState).
Export the hook as a named export: export function useSilenceDetector
```

```
This hook will be called from Interview.jsx like this:
  const { start, stop, reset } = useSilenceDetector(handleSilence, 3000);
  // Every time transcript changes (new speech): call reset()
  // When phase changes to RECORDING: call start()
  // When phase changes away from RECORDING: call stop()
```

PROMPT 7 — Build useInterviewSession.js — localStorage Persistence

```
@workspace  Open file: src/hooks/useInterviewSession.js

Write the useInterviewSession hook that handles ALL localStorage
operations. No other file should ever call localStorage directly.

Use these exact key names:
  const SESSION_KEY = 'iv_session';
  const ANSWER_KEY  = (qId) => `iv_ans_${qId}`;
  const INDEX_KEY   = 'iv_current_index';

Export these functions (NOT as a class — export as a plain hook):

saveSession(sessionId, questions)   — saves session + index=0
loadSession()   — returns { sessionId, questions } or null
                  wrap in try/catch — return null on parse error
saveCurrentIndex(index)   — saves current question index
loadCurrentIndex()   — returns integer (default 0)
saveAnswer(questionId, questionText, questionType, answerText)
    saves: { questionId, questionText, questionType, answerText,
             timestamp: new Date().toISOString() }
loadAnswer(questionId)  — returns stored object or null
loadAllAnswers(questions)   — maps over questions array,
    for each: load stored answer OR return a default object with
    answerText: '[No answer provided]'
clearSession(questions)  — removes SESSION_KEY, INDEX_KEY,
    and all iv_ans_{id} keys for every question in the array
```

5. Phase 3 — Flask Backend

Build the two backend endpoints and the Gemini AI service. Test each endpoint with Postman/Thunder Client before connecting the frontend.

PROMPT 8 — Build the Database Models

```
@workspace  Open file: backend/models.py

Add these three new SQLAlchemy models to the existing models.py file.
Do NOT delete or modify existing models.

MODEL 1: InterviewSession
Columns: id (PK), user_id (FK→users.id), resume_id (FK→resume_data.id),
  status (String 20, default='In Progress'),
  created_at (DateTime, default=utcnow), completed_at (nullable),
  duration_sec (Integer, nullable),
  overall_score (Float nullable), hr_avg_score (Float nullable),
  tech_avg_score (Float nullable), communication_score (Float nullable)
Relationships: questions (backref='session'), evaluation (uselist=False)

MODEL 2: InterviewQuestion
Columns: id (PK), session_id (FK→interview_sessions.id),
  order_num (Integer), question_text (Text), question_type (String 20),
  answer_text (Text nullable), content_score (Float nullable),
  communication_score (Float nullable), relevance_score (Float nullable),
  overall_q_score (Float nullable), ai_feedback (Text nullable),
  strength (String 300 nullable), improvement (String 300 nullable)

MODEL 3: EvaluationResult
Columns: id (PK), session_id (FK unique), evaluated_at (DateTime utcnow),
  model_used (String 50, default='gemini-2.5-flash'),
  overall_score (Float), hr_score (Float), technical_score (Float),
  communication_score (Float), confidence_score (Float),
  structure_score (Float), summary_feedback (Text),
  top_strength (String 400), top_weakness (String 400),
  top_3_recommendations (Text),  # stored as JSON string
  placement_readiness (String 30)

After adding models, also add: db.create_all() or
show me the flask db migrate command to create the tables.
```

PROMPT 9 — Build the Gemini AI Service

```
@workspace  Open file: backend/services/openai_service.py

Write the complete Gemini AI service. I am using Google Gemini
(NOT OpenAI) with the model 'gemini-2.5-flash'.

Install: import google.generativeai as genai
Config:  genai.configure(api_key=os.getenv('GEMINI_API_KEY'))
Model:   MODEL_NAME = 'gemini-2.5-flash'

FUNCTION 1: generate_questions(resume)
  - temperature=0.7, max_output_tokens=1000
  - System instruction: You are an expert technical interviewer at a top
    Indian IT company. Return ONLY valid JSON. No preamble. No backticks.
  - User prompt: build from resume fields:
      Name, Degree/Branch, Skills list, Project summaries, CGPA
  - Question distribution: Q1=HR opener, Q2=Behavioral, Q3=HR closing,
    Q4-Q8=Technical (MUST reference specific skills/projects from resume)
  - Expected return: list of 8 dicts: [{text:..., type:...}]
  - Strip backticks using re.sub before json.loads()
  - Raise exception if result is not a list of 8 items

FUNCTION 2: evaluate_interview(answers)
  - temperature=0.2, max_output_tokens=3000
  - answers is a list of 8 dicts:
    [{questionId, questionText, questionType, answerText}]
  - System instruction: You are an unbiased expert HR evaluator.
    Return ONLY valid JSON. Scores are 0.0-10.0.
  - Build a transcript string from answers and send as one prompt
  - Expected return: the full evaluation dict with overall_score,
    placement_readiness, summary, top_strength, top_weakness,
    recommendations (list of 3), scores dict, question_results list
  - Strip backticks, parse JSON, return dict
  - Wrap entire function in try/except — reraise as ValueError
```

PROMPT 10 — Build /api/interview/start Endpoint

```
@workspace  Open file: backend/routes/interview.py

Write the POST /api/interview/start endpoint.

Requirements:
1. Decorated with @jwt_required()
2. Get user_id from get_jwt_identity()
```

```
3. Get resume_id from request JSON body
4. GUARD: check if an InterviewSession with status='In Progress'
   already exists for this user. If yes: return 409 with the existing
   session_id so frontend can resume it.
5. Load ResumeData by resume_id AND user_id (security check)
   If not found: return 404
6. Call openai_service.generate_questions(resume) in try/except
   If fails: return 503 with error message
7. Create InterviewSession, flush to get the ID
8. Loop through 8 questions, create InterviewQuestion for each,
   flush each to get IDs, build question_list
9. db.session.commit()
10. Return 200 with:
    { session_id: int, questions: [{id, order, text, type}] }

Error responses:
  404: { error: 'Resume not found' }
  409: { error: 'Active session exists', session_id: int }
  503: { error: 'Question generation failed: <message>' }
  400: { error: 'resume_id is required' }
```

TIP: Test with Thunder Client / Postman: POST /api/interview/start with a valid JWT and resume_id. Verify you get 8 questions back.

PROMPT 11 — Build /api/interview/submit-all Endpoint

```
@workspace  Open file: backend/routes/interview.py

Add the POST /api/interview/submit-all endpoint to the same file.

Requirements:
1. @jwt_required() — verify user owns this session
2. Request body: { session_id, answers: [{questionId, questionText,
   questionType, answerText}] }
3. Validation:
   - answers list empty: return 400
   - answers list < 3 items: return 400 with message
   - session not found for this user_id: return 404
   - session.status == 'Completed': return 409 'Already evaluated'
4. Call openai_service.evaluate_interview(answers) in try/except → 503
5. Loop through evaluation['question_results'], find each
InterviewQuestion
   by ID, save all score fields and feedback fields
6. Update InterviewSession: status='Completed', completed_at=utcnow,
   overall_score, hr_avg_score, tech_avg_score, communication_score
```

```
7. Create EvaluationResult row with all holistic scores
8. db.session.commit()
9. Return jsonify(evaluation), 200
   (return the FULL evaluation dict — same as what came from Gemini)

IMPORTANT: import json at the top — use json.dumps() for the
top_3_recommendations field before saving to the Text column.
```

TIP: Test: POST /api/interview/submit-all with a session_id and 8 sample answers. Verify the response has overall_score and question_results.

6. Phase 4 — UI Components

Build the visual components. Each prompt builds one component. All components are simple — they only DISPLAY data passed as props. They contain no logic.

PROMPT 12 — Build BrowserCheck.jsx and MicPermission.jsx

```
@workspace

Write TWO components:

--- COMPONENT 1: src/components/interview/BrowserCheck.jsx ---
Props: { onContinue, onFallback }

On mount, check: const supported = !!(window.SpeechRecognition
  || window.webkitSpeechRecognition)

If supported: show green checkmark icon + text 'Voice Interview Ready'
  + subtext 'Chrome detected. Microphone access will be requested next.'
  + blue button 'Continue to Interview →' (calls onContinue)

If NOT supported: show amber warning icon + text 'Voice Not Available'
  + subtext 'Please open this page in Google Chrome for voice features.'
  + a smaller link-style button 'Continue with text input instead'
    (calls onFallback — this lets Firefox users type answers instead)

--- COMPONENT 2: src/components/interview/MicPermission.jsx ---
Props: { onGranted, onDenied }

Shows: microphone icon (large), title 'Allow Microphone Access',
text: 'The interview needs your microphone. Click the button below,
then click Allow when your browser asks for permission.'

One big button 'Allow Microphone & Continue'.
On click: call navigator.mediaDevices.getUserMedia({ audio: true })
  .then(() => onGranted())
  .catch(() => onDenied())

If denied: show red error card with message and a retry button.

Use only inline styles or CSS modules — no external UI libraries.
```

PROMPT 13 — Build PreBrief.jsx and QuestionCard.jsx

```
@workspace

Write TWO components:

--- COMPONENT 1: src/components/interview/PreBrief.jsx ---
Props: { questions, onBegin }

Shows a pre-interview briefing screen with:
- Title: 'Your Interview is Ready'
- Candidate info summary: total questions, HR questions count,
  Technical questions count (derive counts from questions prop)
- 4 rules (numbered list):
  1. Speak clearly after the countdown ends
  2. Pause for 3 seconds to move to the next question
  3. You can also click 'Next Question' to advance manually
  4. All answers are saved automatically
- Large green button 'Begin Interview' (calls onBegin)

--- COMPONENT 2: src/components/interview/QuestionCard.jsx ---
Props: { phase, question, questionNumber, totalQuestions,
         transcript, countdown, onSkip }

Layout:
  TOP BAR: 'Question {N} of {total}' + progress bar + type badge
    Type badge colors: hr=blue, technical=orange, behavioral=purple

  MIDDLE SECTION (changes per phase):
    phase==='speaking': AI avatar circle with pulsing rings animation
    phase==='countdown': large countdown number with circle animation
    phase==='recording': pulsing red dot + 'Recording...' text
    phase==='saving_answer': green checkmark + 'Answer Saved'

  QUESTION TEXT: large, centered, below the avatar area

  TRANSCRIPT BOX (only when phase==='recording'):
    Light blue background box showing live transcript text
    Placeholder text if transcript is empty: 'Start speaking...'

  BOTTOM: small 'Next Question →' button (calls onSkip)
    Only visible when phase==='recording'

Use CSS keyframe animations for the pulse and countdown circle.
Define them as a <style> tag inside the component (no external CSS file).
```

PROMPT 14 — Build AnswerReview.jsx

```
@workspace  Open file: src/components/interview/AnswerReview.jsx

Props: { questions, sessionId, onSubmit, onReRecord }

This component shows AFTER all 8 questions are answered,
before the user submits for AI evaluation.

Import useInterviewSession hook and call loadAllAnswers(questions).

Layout:
  - Header: green checkmark icon + 'Interview Complete!'
  - Subheader: 'Review your answers before submitting.'

  - For each of the 8 questions, render a card showing:
    * Question number + type badge
    * Question text
    * Answer text (from localStorage via loadAllAnswers)
    * If answerText === '[No answer provided]': show amber warning
      badge 'No answer recorded'
    * Small 're-record Answer' button → calls onReRecord(index)

  - BOTTOM:
    * Small grey text: 'Evaluation typically takes 20-30 seconds'
    * Large green button 'Submit for AI Evaluation →' → calls onSubmit
    * Button must be DISABLED and show 'Submitting...' while loading
      (add a local isSubmitting state, set true on click)

IMPORTANT: The 'Submit' button must only be enabled if at least
3 out of 8 answers have actual text (not '[No answer provided]').
Show a message if fewer than 3 answers are recorded.
```

PROMPT 15 — Build InterviewResults.jsx and ScoreGauge.jsx

```
@workspace

Write TWO components:

--- COMPONENT 1: src/components/results/ScoreGauge.jsx ---
Props: { score, label, size='large' }

A circular score display. Show the score as a large number (e.g. 7.2)
```

```
inside a circle. Color the circle border/background based on score:
  score < 5: red (#B71C1C background tint, red border)
  score 5-6.9: amber (#E65100 tint, amber border)
  score >= 7: green (#2E7D32 tint, green border)
Below the number show the label text.
Implement using CSS border-radius circle, no external library.

--- COMPONENT 2: src/pages/InterviewResults.jsx ---
Props: { results }  (the full evaluation JSON from the API)

Layout (in this exact order, top to bottom):
  1. Header bar: 'Your Interview Results'
  2. ScoreGauge: overall_score + placement_readiness badge
     Badge colors: Not Ready=red, Needs Work=amber,
     Almost Ready=blue, Ready=green, Exceptional=purple
  3. Score row: 5 small ScoreGauge components (hr, technical,
     communication, confidence, structure scores)
  4. AI Summary box: results.summary text in a styled card
  5. Two columns: Top Strength (green card) + Top Weakness (amber card)
  6. Recommendations: numbered list of results.recommendations
  7. Question breakdown: 8 expandable accordion cards
     Each card: Q text, answer text, per-score numbers, feedback,
     strength (green), improvement (amber)
  8. Download button: 'Download PDF Report' — calls
     generateInterviewPDF() from utils/pdfGenerator.js

Import and include <ReportTemplate results={results} /> at the bottom
(it will be hidden by default — the PDF function reveals it).
```

7. Phase 5 — Wire Everything Together

These are the two final and most complex prompts. They assemble all hooks and components into the working state machine.

PROMPT 16 — Build the Complete Interview.jsx State Machine

```
@workspace  Open file: src/pages/Interview.jsx

Write the complete Interview.jsx component. This is the main controller.
Study the attached blueprint document carefully before writing this.

PHASE CONSTANTS (define as a frozen object at the top):
PHASES = { BROWSER_CHECK, MIC_PERMISSION, LOADING_QUESTIONS,
  PRE_BRIEF, SPEAKING, COUNTDOWN, RECORDING, SAVING_ANSWER,
  REVIEW, SUBMITTING, RESULTS, ERROR }

PROPS: { resumeId }

HOOKS to import and use:
  useTTS         → { speak, stopSpeaking }
  useSTT         → { transcript, isRecording, isSupported,
                     sttError, startRecording, stopRecording,
                     resetTranscript }
  useSilenceDetector(handleSilence, 3000)
               → { startSilenceDetection, stopSilenceDetection,
                   resetSilenceTimer }
  useInterviewSession → all session functions

STATE: phase, questions, currentIdx, sessionId, results,
       error, countdown (for 2s display), textFallback (bool)

EFFECTS:
  1. On mount: call loadSession(). If session exists, restore
     questions/sessionId/currentIdx. Set phase=MIC_PERMISSION
  2. When transcript changes AND phase===RECORDING:
     call resetSilenceTimer()
  3. When sttError===='not-allowed': setError + setPhase(ERROR)

FUNCTION runQuestion(index) — ASYNC:
  If index >= questions.length: setPhase(REVIEW) and return
  1. resetTranscript(), saveCurrentIndex(index)
  2. setPhase(SPEAKING), call stopSpeaking()
     AWAIT speak(`Question ${index+1}. ${question.text}`)
  3. setPhase(COUNTDOWN), countdown from 2 to 0 using setInterval,
     AWAIT a Promise that resolves when countdown hits 0
```

```
  4. setPhase(RECORDING), startRecording(), startSilenceDetection()

FUNCTION finalizeAnswer() — called by silence AND manual skip:
  1. stopSilenceDetection(), get finalText=stopRecording()
  2. saveAnswer(q.id, q.text, q.type, finalText||'[No answer provided]')
  3. setPhase(SAVING_ANSWER)
  4. setTimeout 800ms → increment currentIdx, call runQuestion(nextIdx)

FUNCTION startInterview() — ASYNC:
  setPhase(LOADING_QUESTIONS)
  Call POST /api/interview/start via interview.js API service
  On success: setSessionId, setQuestions, saveSession,
setPhase(PRE_BRIEF)
  On error: setError with message, setPhase(ERROR)

FUNCTION submitInterview() — ASYNC:
  setPhase(SUBMITTING)
  Call loadAllAnswers(questions), POST /api/interview/submit-all
  On success: setResults, clearSession(questions), setPhase(RESULTS)
  On error: setError message (note: answers still saved), setPhase(ERROR)

RENDER: return the correct component for each phase.
For the question phases (SPEAKING/COUNTDOWN/RECORDING/SAVING_ANSWER)
return <QuestionCard> with all needed props.
```

TIP: This is the longest and most complex file. After Copilot writes it, carefully check: 1) all imports are correct, 2) all hook names match exactly, 3) the async/await chain in runQuestion is complete.

PROMPT 17 — Build pdfGenerator.js and ReportTemplate.jsx

```
@workspace

Write TWO files:

--- FILE 1: src/utils/pdfGenerator.js ---

import jsPDF from 'jspdf';
import html2canvas from 'html2canvas';

Export an async function: generateInterviewPDF(candidateName,
sessionDate)

Logic:
1. Get element: document.getElementById('interview-report-pdf')
```

```
2. If not found: console.error and return early
3. Set element.style.display = 'block'
4. Call html2canvas(el, { scale:2, useCORS:true, backgroundColor:'#fff',
   logging:false })
5. Set element.style.display = 'none'
6. Create jsPDF({ orientation:'portrait', unit:'mm', format:'a4' })
7. Calculate imgH = (canvas.height * pageWidth) / canvas.width
8. Add first page image
9. While heightLeft > 0: addPage(), addImage at offset position
10. Save as: Interview_Report_{safeName}_{sessionDate}.pdf
    where safeName = candidateName.replace(/[^a-zA-Z0-9]/g, '_')

--- FILE 2: src/components/results/ReportTemplate.jsx ---
Props: { results, userName, sessionDate }

A div with id='interview-report-pdf' and style={{ display:'none' }}
Uses only INLINE styles (no CSS classes — html2canvas needs inline
styles)
Width: 800px fixed, padding 24px, white background, Arial font

Contains (all with inline styles):
  - Navy blue header bar: title + candidate name + date
  - Overall score large number (color based on score)
  - Placement readiness badge
  - Simple HTML table for 5 dimension scores
  - Summary paragraph, top strength, top weakness
  - Numbered recommendations
  - For each question: bordered div with Q text, answer, scores, feedback
  - Footer: 'Generated by AI Pre-Placement Trainer'

IMPORTANT: Do NOT use any className or external CSS.
Every style must be a JavaScript inline style object.
```

PROMPT 18 — Final Wiring — API Service and App Router

```
@workspace

Complete TWO final tasks:

--- TASK 1: src/services/api/interview.js ---

Write the API service file. Import axiosInstance from './axiosInstance'
(which already has the JWT token interceptor).
```

```
Export these async functions:

startSession(resumeId):
  POST /api/interview/start
  Body: { resume_id: resumeId }
  Returns: response.data (contains session_id, questions)

submitAll(sessionId, answers):
  POST /api/interview/submit-all
  Body: { session_id: sessionId, answers: answers }
  Returns: response.data (contains full evaluation)

getSessionHistory():
  GET /api/interview/history
  Returns: response.data

Both POST functions must throw errors with:
  error.response?.data?.error || 'An error occurred'
so Interview.jsx can display the correct message to users.

--- TASK 2: Add Interview route to the React Router ---

@workspace  Open file: src/App.jsx (or wherever your routes are)

Add a new protected route:
  path: /interview
  component: Interview
  Pass resumeId as a prop: get it from the logged-in user's stored
  resume data (from localStorage or Redux store — wherever you
  currently store user/resume data)

Also add a 'Start Interview' button on the Dashboard page
that navigates to /interview when clicked.
```

TIP: This is the final prompt. After this, run your app and test the full flow end to end.

8. Final Testing Checklist

After all 18 prompts are done, run through this checklist. Only check an item when you have personally tested it, not just when the code looks correct.

Phase A — Voice Flow (Chrome Desktop)

[ ] Open /interview in Chrome — BrowserCheck shows green ✓
[ ] Click 'Continue' → MicPermission screen appears
[ ] Click 'Allow Microphone' → browser permission popup appears
[ ] Grant permission → PreBrief screen shows 8 questions summary
[ ] Click 'Begin Interview' → Q1 screen loads
[ ] TTS reads Question 1 aloud (you hear audio)
[ ] After TTS finishes: countdown '2...1' displays correctly
[ ] Microphone starts automatically — recording indicator appears
[ ] Speak for 5 seconds → transcript appears in the blue box
[ ] Stop speaking for 3 seconds → 'Answer Saved' flash appears
[ ] Q2 loads automatically and TTS reads it aloud
[ ] Complete all 8 questions without typing anything
[ ] AnswerReview page shows all 8 answers from localStorage
[ ] Click 'Submit' → loading screen 'AI is evaluating...'
[ ] Results page loads with overall score and all feedback
[ ] Click 'Download PDF Report' → PDF file downloads
[ ] Verify PDF has all 8 questions with scores and feedback

Phase B — Edge Cases

[ ] Refresh browser mid-interview (on Q4) → page reloads to Q4
[ ] Use 'Re-record' on AnswerReview → re-recording works
[ ] Open in Firefox → browser warning shown (NOT a crash)
[ ] Text fallback in Firefox → user can type answers and submit
[ ] Check Network tab → only ONE API call made during submission
[ ] Start interview, abandon, start again → no 409 error (old session cleared)

Quick Bug Fixes for Common Copilot Errors

ERROR: 'getVoices() returns []' → Add speechSynthesis.onvoiceschanged listener
ERROR: 'STT stops after 30s' → Check that rec.onend restarts recognition when isRecordingRef=true
ERROR: 'PDF is blank white' → The report div must have display:block before html2canvas runs
ERROR: 'Silence fires immediately' → Check that startSilenceDetection() is only called AFTER startRecording()
ERROR: 'Double question read' → Ensure stopSpeaking() is called at start of every runQuestion()
ERROR: 'JSON parse error from Gemini' → Add re.sub(r'```json|```', '', raw) before json.loads()

ERROR: '401 Unauthorized on submit' → Check axiosInstance is setting Authorization: Bearer <token>

End of Document — GitHub Copilot Prompt Guide | AI Interview Module

YSPM's Yashoda Technical Campus, Satara | AI & DS Department | 2025-26